JADE: Joint Architecture-Dataflow Exploration for LLM Inference on Heterogeneous In- and Near-Memory Computing Systems

Yimin Wang, *Graduate Student Member, IEEE*, Zhen Wu, *Graduate Student Member, IEEE*, Yue Jiet Chong, *Graduate Student Member, IEEE*, and Xuanyao Fong, *Member, IEEE*

***Abstract*—Large Language Model (LLM) inference challenges memory/computing organization and dataflow optimization on traditional hardware stacks due to its various attention mechanisms and autoregressive structure. In-memory computing (IMC) and near-memory computing (NMC) architectures have emerged as potential solutions to reduce the memory burden in LLMs. However, efficient LLM orchestration on IMC/NMC, as well as dedicated IMC/NMC architecture optimization, are interrelated, encompassing a hardware-software co-design space of a myriad of parameters, which still remains largely underexplored. This work provides a joint architecture-dataflow exploration (JADE) framework that covers both architectural factors on computing/memory/communication, and algorithmic factors on LLM dataflow. We identify key configurations in four categories: computing and memory organization, interconnection topology, matrix multiplication (MM) unrolling and mapping, and attention mechanism. A unified MM representation enables flexible unrolling and mapping on various architectural organizations. Dataflow is routed based on the collective communication patterns inherent in the distributed LLMs and the features of common topologies. Finally, case studies show the design tradeoffs between key design options and provide insights into both homogeneous and heterogeneous IMC/NMC design for LLMs.**

***Index Terms*—Large language model, in-memory computing, near-memory computing, design space exploration, heterogeneous architecture.**

I. Introduction

THE rapid growth in the scale of large language models (LLMs) is driving hardware accelerator design scaling up and out to meet increasing memory and compute demand. Traditional shared-memory architectures such as GPUs have continued to deliver performance gains, but memory bandwidth remains a crucial challenge. Emerging spatial accelerators, with highly distributed local memory and compute units, have demonstrated high-performance potential and scalability toward even wafer scale, including the Cerebras Wafer-Scale Engine (WSE) [1], Tesla's Dojo [2], and coarse-grained reconfigurable arrays (CGRAs) [3], [4].

Meanwhile, in-memory computing (IMC) has emerged as a promising approach to further mitigate the memory bandwidth bottleneck. IMC efficiently performs matrix multiplications (MMs) by statically storing weight matrices and executing computations directly within memory. This paradigm is particularly well-suited for weight projections in attention layers and feed-forward networks (FFNs) of LLM. However, other components of LLM inference involving run-time generated data require more flexible processing, which can be supported by complementary near-memory computing (NMC) resources. This necessitates a joint utilization of IMC and NMC architectures [5], [6], [7], [8], [9], [10], and a tightly-coupled IMC/NMC design becomes essential to effectively support the hybrid data-stationarity patterns inherent in LLM workloads.

While IMC and NMC have shown promise individually, their potential as tightly-coupled compute resources in the spatial accelerators for LLM workloads remains underexplored. It is crucial to determine the proper configurations for the hybrid IMC/NMC architecture and efficient orchestration of LLM dataflow on the spatial arrays to achieve energy efficiency and performance. This is challenging due to the enlarged design space that comprises multiple design dimensions spanning the hardware architecture, LLM models, and dataflow scheduling, especially under the growing model sizes and extended context windows. Existing LLM/transformer accelerators [8], [11], [12], [13], [14] mostly focus on a fixed architectural setup with little flexibility on the resource organization and interconnection. Existing memory-centric dataflow exploration frameworks [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25] are predominantly tailored for conventional neural networks (NNs) and fall short in addressing the design challenges of LLMs.

This paper presents a ***J***oint ***A***rchitecture-***D***ataflow ***E***xploration (JADE) framework to explore LLM inference workload on the spatial arrays with hybrid IMC and NMC, considering various architectural and algorithmic aspects. We identify the key design factors that impact the overall performance and classify them into four categories: i) computing and memory organization, ii) interconnection topology, iii) attention mechanism, and iv) MM unrolling. In this work, a unified MM representation enables flexible

Received 31 August 2025; revised 27 December 2025 and 18 March 2026; accepted 26 March 2026. Date of publication 30 March 2026; date of current version 15 June 2026. This work was supported in part by the National Research Foundation, Prime Minister's Office, Singapore, under its Competitive Research Program, under Grant NRF-CRP24-2020-0002 and Grant NRF-CRP24-2020-0003; and in part by the National University of Singapore through the Microelectronics Seed Grant under Grant FY2024. This article was recommended by Guest Editor A. Bosio. *(Corresponding author: Xuanyao Fong.)*

The authors are with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117583 (e-mail: yimin.wang@u.nus.edu; kelvin.xy.fong@nus.edu.sg).

Digital Object Identifier 10.1109/JETCAS.2026.3679018

2156-3357 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

unrolling and mapping on various architectural organizations. Dataflow is routed based on the collective communication patterns inherent in the distributed LLMs, and the features of common topologies, *i.e.*, mesh and tree. Evaluated with a wide range of LLM models and architecture templates, JADE demonstrates significant improvement compared to the state-of-the-art and unveils insightful design tradeoffs that cater to various application scenarios. The framework provides a unified evaluation approach for existing IMC/NMC-based LLM accelerators because of its wide coverage of the multi-dimensional design space. The key contributions of this work are summarized as follows:

- This work largely extends the IMC/NMC LLM accelerator design space with consideration of the full computing/memory/communication stack, which enables a direct comparison between homogeneous and heterogeneous designs on various levels.
- A unified MM tiling and unrolling representation compatible with various hardware architectures is introduced. The mapping of tiled MMs on hardware is aware of the IMC/NMC difference and the operation flows in LLM attention heads.
- The fine-grained dataflows are routed based on their collective communication patterns and the interconnection topologies, which bridges the dataflow exploration and architectural exploration.
- Evaluation results show the design tradeoffs between key design options and provide insights into both homogeneous and heterogeneous IMC/NMC design for LLMs, which is critical for the future scale-up architecture design for ever-growing LLMs.

The rest of the article is organized as follows. Section II introduces the basics of IMC/NMC-based LLM accelerators and key design factors defining the design space. Section III presents the main steps in the JADE framework: virtual mapping, physical mapping, and dataflow generation. Section IV elaborates the corresponding hardware architecture design. Section V shows the evaluation and discusses the design tradeoffs and insights. Finally, Section VI concludes this article.

II. Preliminaries

A. IMC/NMC-Based Accelerators

Recent commercial LLMs, such as GPT [26] and the Llama series [27], [28], [29], [30], are predominantly decoder-only Transformers, as shown in the top left of Fig. 1. Each decoder layer comprises an attention layer and fully connected layers, which spawn a series of successive MMs and matrix-vector multiplications (MVMs). Despite the pre-trained weights, successive MMs/MVMs generate a large amount of intermediate dynamic data. In this paper, we name the matrix multiplication ($A \cdot B$) with a dynamic matrix A and a static matrix B as Dynamic-Static Matrix Multiplications (DSMMs) and that with both dynamic matrices A and B as Dynamic-Dynamic Matrix Multiplications (DDMMs). The DSMMs and DDMMs are empirically assigned to IMC and NMC, respectively, as shown in Fig. 1.

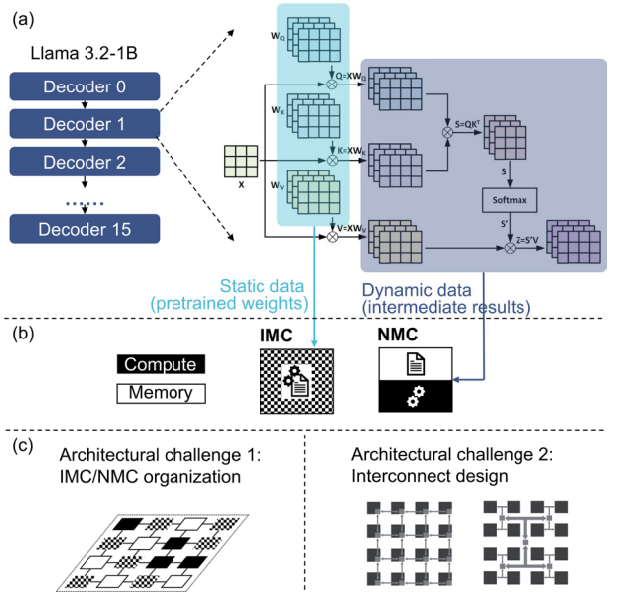


Fig. 1. (a) The basic LLM structure, (b) the data stationarity-based MM assignment on the IMC/NMC system, and (c) the primary challenges in hybrid IMC/NMC system design.

Existing memory-centric dataflow exploration frameworks [15], [16], [17], [18], [19], [20], [21], [23], [24] are predominantly tailored for NNs and fall short in addressing the challenges of LLMs. This is exacerbated by several key challenges: i) LLMs require a flexible allocation of MMs between IMC and NMC based on the stationarity of data; ii) LLM inference exhibits significantly different matrix sizes and cache access patterns between the prefill and autoregressive decode stages; and iii) matrix partitioning in LLMs increases the number of partial results that need to be aggregated by an order of magnitude due to the substantial size of the matrices involved.

Though prior works have shown IMC/NMC-based systems for LLM acceleration, most only support model-specific dataflow with custom architectural design [5], [6], [7], falling short in terms of system and dataflow flexibility to diverse models. AllSpark [11] and Energon [12] focus on the transformer mapping and scheduling on an NMC-only array organized homogeneously in the 2D mesh topology. LEAP [10] presents a 2D mesh array architecture with evenly distributed IMC and NMC resources with dataflow and parallelism optimization for LLM acceleration. HALO [8] studies the transformer scheduling on the chiplet level to minimize the inter-layer communication overhead while the dataflow and architecture within a chiplet is unexplored. Theseus [13] and WaferLLM [14] propose design solutions for wafer-scale LLM accelerators with conventional digital computing units, lacking support of the emerging IMC or NMC elements.

Hence, this work is motivated to explore the hybrid IMC/NMC LLM accelerator design in an extended design space that covers flexible computing/memory/communication organization on various levels. The comparison of this work with existing works is shown in Table I. The previous works may not include i) the various constraints in IMC/NMC (*e.g.*,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON TO STATE-OF-THE-ART LLM/TRANSFORMER ACCELERATORS

	① PE level	① Tile level	② Interconnection	③ Target model	④ MM mapping
JADE (**This work**)	IMC & NMC	Hetero. & Homo.	Flex. mesh & tree	Decoder-only LLM	Unified representation
HALO [8]	IMC & Digital	Heterogeneous	Fixed mesh & tree	Small, BERT Limited length	/
Energon [12]	NMC	Homogeneous	2D mesh	Small, Vision transformer	/
AllSpark [11]	NMC NVDLA-style	Homogeneous	2D mesh	Small, Vision transformer Local attention in ViT	/
Theseus [13]	NMC	Homogeneous	2D mesh	Decoder-only LLM	/
WaferLLM [14]	NMC	Homogeneous	2D mesh	Decoder-only LLM	/
LEAP [10]	IMC & NMC	Homogeneous	2D mesh	Decoder-only LLM	/

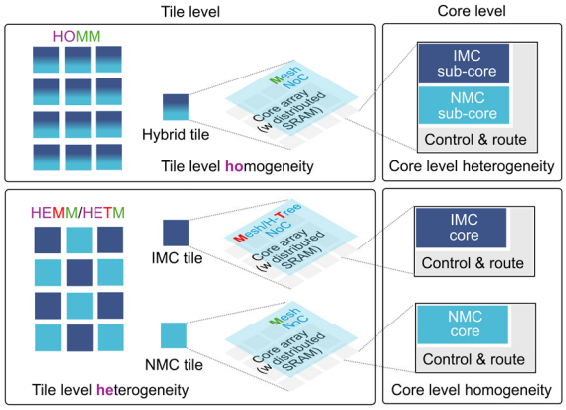


Fig. 2. The hardware hierarchy and the categorization of homogeneity and heterogeneity. Note that the placement of the IMC tiles and the NMC tiles in HE architecture is not fixed. The chessboard placement in the above figure is a toy example.

the unsupported constraints of IMC in Energon, AllSpark, Theseus, and WaferLLM); ii) heterogeneous/homogeneous setup (*e.g.*, homogeneous setup in HALO and heterogeneous setup in the other works in Table I); or iii) the interconnection topologies (*e.g.*, the missing consideration of tree topology in Energon, AllSpark, Theseus, WaferLLM, and LEAP). Consequently, the design space explored in these prior studies represents only a subset of the comprehensive scope in this work.

B. Design Space Construction

We classify the key design factors into four categories, which span across the hardware hierarchies. We consider the hardware organized in two levels: core and tile [13], as shown in Fig. 2, where the core hosts the elementary IMC or NMC unit, and the tile comprises an array of IMC or NMC, which is used for a layer or a partial layer.

1) Computing and Memory Organization: We specify the differentiated setup for IMC and NMC on both the tile and core levels: the system can be tile-level homogeneous and core-level heterogeneous (referred to as homogeneous architecture HO), or tile-level heterogeneous and core-level homogeneous (referred to as heterogeneous architecture HE), as shown in Fig. 2. A core can be dedicated to either IMC or NMC units (core-level homogeneous) or host both IMC and NMC units (core-level heterogeneous).

2) Interconnection Topology: Mesh and tree topologies are considered for intra-tile connections, given their common applications in multi-core architecture and memory organization, respectively. This work only considers the tree topology in connecting IMC tiles, where the broadcast and reduction can be efficiently performed on different hierarchy levels. Whereas the DDMMs in NMC feature more point-to-point communications, which do not suit the tree topology. Therefore, the architecture under consideration is confined to HOMM, HEMM, and HETM, as shown in Fig. 1, where the last two letters stand for the intra-tile interconnection topology for IMC followed by that for NMC resources.

3) Attention Mechanism: This category considers the attention type and kernel fusion options. Common attention types are multi-head attention (MHA) and grouped-query attention (GQA). In traditional MHA, each K/V head has a corresponding Q head. In GQA, K/V heads are shared in a group of Q heads, which intends to reduce the burden on computing and memory overhead. The kernel fusion mechanism, *e.g.*, FlashAttention [31], was originally developed to minimize the off-chip memory access on the GPU when performing attention layers. In this work, we evaluate its effectiveness on the IMC/NMC-based systems.

4) MM Unrolling and Mapping: Unrolling and mapping the matrices onto the IMC or NMC cores is critically important in the array-based accelerators, interrelated to the data reuse and reduction. We unify the unrolling of both DDMM and DSMM into multiple nested loops, and correlate them to the hardware dimensions in both mesh and tree topologies.

III. SYSTEM-LEVEL ORCHESTRATION

JADE comprises both the system-level orchestration framework and the corresponding hardware architecture design. This section presents the system-level aspects, including virtual mapping, physical mapping, and dataflow generation.

A. Framework Overview

The overview of the orchestration framework of JADE is shown in Fig. 3. JADE takes in three configuration files, which specify the information and setup of the model, hardware, and explorer. The model configuration file contains the basic LLM information: the number of layers, the layer sizes, the sequence length limit, and the attention type (MHA or GQA). The hardware configuration file specifies the architectural template (HO and HE), the memory/computing capacity, and

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

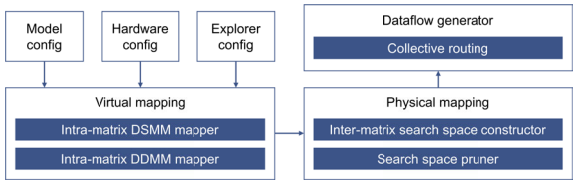


Fig. 3. JADE orchestration framework covers three key parts: virtual mapping, physical mapping, and dataflow routing.

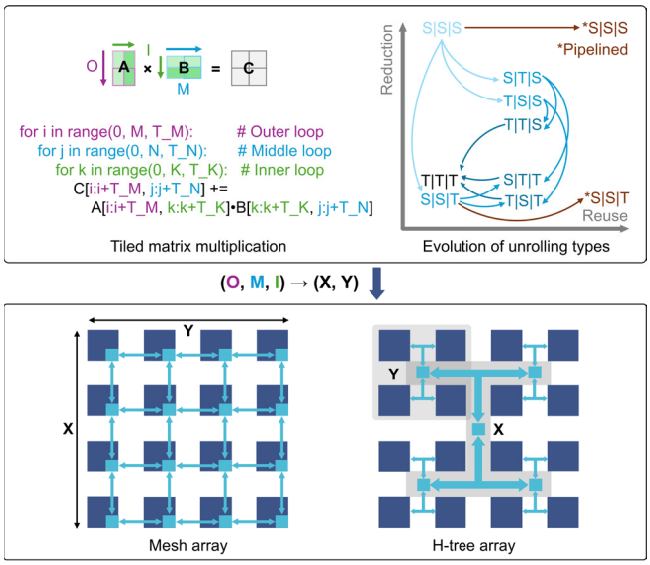


Fig. 4. Large MM tiling and spatial unrolling on 2D hardware arrays with mesh or H-tree topologies.

communication capacity. The explorer configuration file sets up the design space constraint and the kernel fusion option.

The framework runs through three main steps after initialization, based on the input files: virtual mapping, physical mapping, and dataflow generation. The virtual mapping handles the tiling and unrolling of each DSMM/DDMM operation involved in the LLM. The physical mapping covers the inter-matrix placement of tiled and unrolled DSMMs on IMC and DDMMs on NMC. The dataflow generator finally routes the input/output and intermediate results transfer based on the physical placement determined in the previous mapping steps, and evaluates several key metrics: communication cost and inference latency. These three key components will be introduced in the rest of the section.

B. Virtual Mapping

Virtual mapping tiles the large matrices in LLM inference to manageable sizes and unrolls the multiplication loops onto an ideal 2D hardware array, before considering the physical inter-matrix placement in the next step. MM involves two matrices, and each matrix is partitioned along two dimensions, which results in three nested loops as shown on the top left of Fig. 4. The partitioned matrices in each loop can be unrolled either spatially along the hardware dimensions or temporally by streaming in.

1) Representation in Mesh Topology: Assuming a 2D array hardware architecture, the spatial unrolling can align the horizontal (X) or vertical (Y) dimensions as shown in the bottom left of Fig. 4. Loop tiling techniques in this paper are denoted as O|M|I → X|Y|T which represent that the outer loop, the middle loop, and the inner loop are unrolled spatially along the X, Y dimensions, and temporally, respectively. This notation aligns with the existing dataflow representation for convolutions [18]. The tiling size on spatially unrolled dimensions is determined by the computation and memory resources in each core. The tiling size on temporally unrolled dimensions is dependent on the interconnection bandwidth, which is minimally a single row/column of the tiled matrix under the bandwidth bound.

2) Representation in H-Tree Topology: Consider a hardware array organized in an H-tree topology with a total of L levels. At a chosen level L_x, there are 4^{L_x} nodes (referred to as core groups), and each node encompasses 4^{L-L_x} leaf nodes (cores). In this work, the core groups and the cores within each group in the H-tree are interpreted as the X and Y dimensions, respectively, as shown in the bottom right of Fig. 4. In the hybrid H-tree and 2D mesh topology where sub-trees are connected by a 2D mesh on top, the mesh serves as the L_x level, which is the X dimension, and the sub-trees are the Y dimension.

3) Unrolling Categories: Mathematically, there are eight different unrolling categories based on the possible combinations of spatial (S) or temporal (T) unrolling on the three loops. S represents spatial unrolling on X or Y, which have been defined in both mesh and tree topology. The evolution of the unrolling categories regarding the resource sufficiency is shown in the top right of Fig. 4. S|S|S stands for the resource-overprovisioned scenarios, which have been demonstrated in GPU architectures [32], [33]. S|S|T, S|T|S, and T|S|S (*i.e.*, two dimensions unrolled spatially) are commonly considered to process hardware arrays with regular 2D topologies. Whereas S|T|T, T|S|T, T|T|S, and T|T|T refer to resource-limited application cases in which the temporal unrolling dominates.

4) Stationarity-Based Unrolling Strategy: Due to the different temporal and spatial constraints in IMC and NMC, their considerations for unrolling are different.

a) DSMM unrolling in IMC: DSMMs appear in the projection step in the attention layers. They fit well with the IMC cores due to the stationarity of the weight tensors $\mathbf{W_Q}$, $\mathbf{W_K}$, $\mathbf{W_V}$, and $\mathbf{W_O} \in \mathbb{R}^{D\times D}$, where D is the embedding dimension. DSMMs adopt the T|S|S scheme when the IMC resources are sufficient, meaning the middle and inner loops corresponding to the weight tensors are fully unrolled spatially along the IMC PE array. Thus, the tiled weight tensors are stored in the crossbars. The number of crossbar arrays required by each matrix after tiling is $\lceil \frac{D}{C} \rceil^2$, where D is the embedding dimension and C denotes the crossbar width/height.

b) DSMM unrolling in NMC: Although NMC mainly handles DDMMs, it can also support DSMMs when IMC resources are insufficient. Dynamic matrices in the attention include $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V} \in \mathbb{R}^{D\times S}$, and $\mathbf{S} \in \mathbb{R}^{S\times S}$. While S|S|T, S|T|S, and T|S|S unrolling can all suit NMC arrays due to their 2D spatial unrolling, this work considers S|S|T and T|S|S to keep one matrix stored in the scratchpad in the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

NMC cores and then broadcast in the other matrix, which achieves high utilization of both scratchpads and computing units in NMC arrays.

Compared to existing frameworks on unrolling representation, though some [16], [34] have covered a wide range of MM unrolling, this work refines the unrolling options to IMC and NMC, considering their primary difference on data stationarity, and meanwhile extends the underlying hardware interconnection from predominant mesh topology to H-tree topology as well. Compared to prior works on LLM acceleration [10], [11], [12], [14], this unified representation unlocks the design space by unleashing the unrolling options, interconnection topologies, and flexible IMC/NMC allocation.

C. Physical Mapping

Based on the tiled and unrolled matrices from virtual mapping, the next step is to physically allocate the spatially unrolled matrices to certain cores and the temporally unrolled matrices to certain locations in the memory hierarchy (distributed scratchpad, local buffer, or main memory).

1) Intra-Matrix Mapping: The `S` in unrolling can be mapped to `X` or `Y` based on the resource availability. When the available resources on the hardware are insufficient for the spatial unrolling, the loop can also be unrolled in both temporal and spatial dimensions (`XT` or `YT`), meaning new data can be fed in temporally but at each time the data are spatially distributed. In tree topologies, `X` or `Y` can be flexibly fit to the tiled matrix size by adjusting the L_x level. In mesh topologies, the `X` or `Y` for each matrix is also dependent on the core allocation among the different matrices, which is to be elaborated in the following inter-matrix mapping.

2) Inter-Matrix Mapping: This step determines the relative placement of the various matrices on the mesh array. The inter-matrix placement is not considered in the H-tree since the sub-trees at the L_x level are topologically identical and have no impact on the dataflow and utilization. Inter-matrix mapping is also distinct in `HE` and `HO` architectures: the inter-matrix mapping of DSMMs and DDMMs in `HE` are decoupled due to the physical separation of IMC/NMC array, whereas that in `HO` are coupled to each other. In `HO` architectures, the inter-matrix mapping for DSMMs ($\mathbf{W_Q/W_K/W_V/W_O}$) is determined first, and then the mapping for DDMMs (**Q/K/V/O**) is aligned to the region of their weight tensors. This alignment helps direct K/V cache management in their local region and also avoids the imbalanced matrix sizes in GQA.

3) Mapping Space Generation: This step is focused on generating the possible inter-matrix placements on mesh topologies. It follows three heuristic constraints regarding the regularity and proximity of the placement: i) Sub-matrices originating from the same weight matrix must be placed within a spatially proximate region; ii) This region should have a rectangular shape to facilitate regular dataflows and reduce routing complexity; and iii) The sub-matrices within this region should be ordered in a row-major or column-major fashion. As shown at the top of Fig. 5, four key factors are abstracted, and all the possible layouts that satisfy the model layer size and the available resource limit, considering the three heuristic constraints, will be generated.

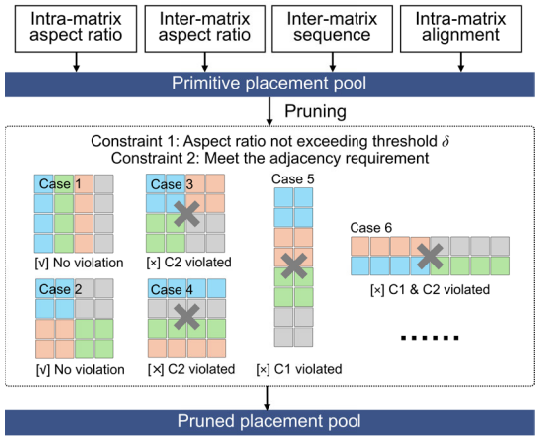


Fig. 5. Inter-matrix mapping design space generation and pruning for mesh topologies.

4) Mapping Space Pruning: To enable efficient design space exploration, this work also includes a pruning step to further discard the infeasible cases before evaluating all of them. Two key constraints are considered to determine the feasibility of a design case. Constraint 1 (C1) is that the overall aspect ratio of the physical layout on tiles should not exceed a predefined range. δ is a hyperparameter specifying the extremum aspect ratio, and the physical layout whose aspect ratio is within $\left[\delta, \frac{1}{\delta}\right]$ can be kept for the subsequent evaluation. Constraint 2 (C2) is that the adjacency of the matrices should align with the original operation flow. For example, the area allocated to **Q** and **K** should be physically adjacent to efficiently perform the $\mathbf{QK^T}$ calculation, and the **O** area should be adjacent to the outward border connecting to the next layer. Architectural candidates violating either C1 or C2 are pruned, and only those satisfying both constraints are admitted to the placement pool. These scenarios are illustrated in Fig. 5: Cases 1 and 2 represent feasible designs with no violations. Conversely, Case 3 and Case 4 are pruned due to C2 violations regarding the isolation of **K**/**Q** and **V**/**O**. Case 5 is excluded for exceeding the aspect ratio threshold (C1), and Case 6 is discarded for violating both constraints.

5) Kernel Fusion Considerations: We consider both the attention as originally proposed in [35], which we refer to as original attention in the rest of the paper, and the kernel-fused attention known as FlashAttention [31]. The key difference between them is the total volume of the intermediate matrices. The complete attention score matrix $\mathbf{S} \in \mathbb{R}^{S\times S}$ needs to be stored in the original attention, whereas FlashAttention replaces it with an $\mathbf{O}^* \in \mathbb{R}^{S\times D}$. FlashAttention [31] fuses the two successive MMs–$\mathbf{QK}^T$ and $\mathbf{S'V}$ into unified nested loops, which means the matrix **S** does not need to be stored. In this work, the original **S** is stored in the scratchpad of Q, and the additional $\mathbf{O}^*$ is stored in the scratchpad of O. FlashAttention originally reduces the amount of data generated at a time in the on-chip memory of GPUs, and its effectiveness on the IMC/NMC-based distributed architecture will be discussed in Section V.

6) Inter-Layer Placement: A decoder comprises an attention layer and FFNs (*e.g.*, a decoder in Llama models has

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

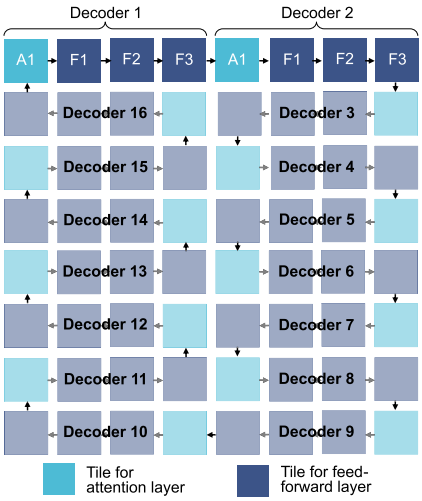


Fig. 6. Inter-layer placement for the whole LLM, exemplified by the Llama 3.2-1B model.

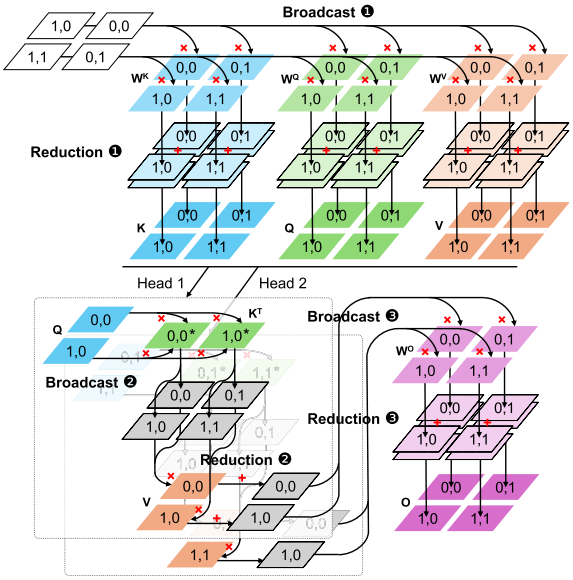


Fig. 7. Collective communication overhead incurred by tiling and unrolling key MM in an example layer of the original attention. Note that the nonlinear operations are not included in this illustration.

three FFN layers). The hidden dimension in the FFN layer is approximately four times the embedding dimension, which means the IMC amount required for an FFN layer is the same as an attention layer. The inter-layer placement follows a serpentine topology for efficient decoding dataflow, as shown in Fig. 6. This placement is friendly to the autoregressive decode phase, which can efficiently relay the latest output token generated by the ending decoder to the beginning decoder.

D. Dataflow Generation

The described virtual mapping and physical mapping have mapped the tiled matrices and intermediate operations spatially and temporally across the array. This introduces extra communication overhead for broadcasting partitioned input

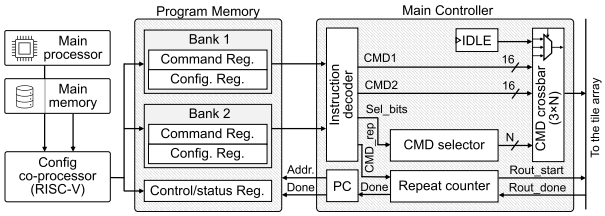


Fig. 8. The system-level architecture of JADE includes program memory, main controller, and the tile array.

and reducing partial results, with an example of a fully tiled attention layer shown in Fig. 7. The dataflow generation step jointly optimizes the communications based on the collective behaviors of data movement and the interconnection topologies. The detailed collective communication patterns supported on the hardware will be introduced in Section IV.

IV. Hardware Architecture

This section details the corresponding hardware architecture design. Section IV-A presents an overview of the high-level architecture. Section IV-B introduces the computation resources, including IMC and NMC cores. Section IV-C details the intra-tile and inter-tile interconnection topologies. Section IV-D elaborates the common collective communication patterns required in the LLM inference.

A. Overall Architecture

The overall architecture comprises program memory, control logic, and a tile array, as illustrated in Fig. 8. The co-processor populates the program memory with instructions consisting of three primary components: i) the command, which configures the operating modes of the core and its associated router; ii) selection bits, which specify the target tile and core IDs; and iii) a repetition count, which defines the number of loop iterations for the current command. To minimize system idle time, the program memory utilizes a double-bank design. This allows for concurrent operation: while the main controller reads and decodes instructions from one bank, the co-processor programs the other. Based on these instructions, the tile array is optionally configured into one of three modes: `HOMM`, `HEMM`, or `HETM`.

B. Computation

1) IMC Core: The IMC units are based on memristive memory arrays, which store the pre-trained weight matrices and perform all DSMMs. For our system-level estimations, we adopt RRAM-based technology [36] as the baseline. Notably, because the JADE framework focuses primarily on mapping-dataflow explorations for LLM inference, the high-level mapping strategies remain independent of the underlying IMC technology. Consequently, the IMC units can be abstracted as closed boxes within the framework.

2) NMC Core: Each NMC core contains a scratchpad memory and multiply-accumulate (MAC) units. The scratchpad stores dynamic data, including **K** and **V**, which persist throughout the full inference cycle, as well as **Q** and **O**, which

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

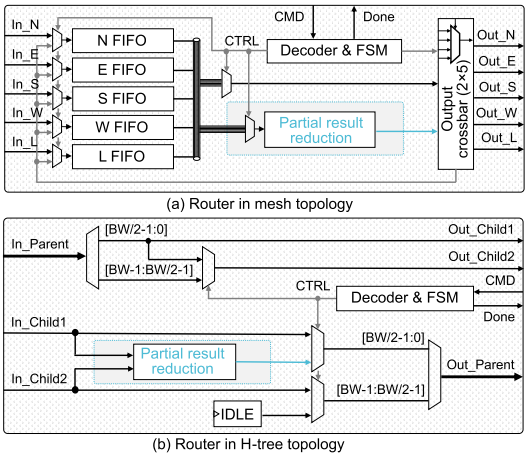


Fig. 9. Router architecture with partial result reduction in (a) mesh and (b) H-tree topology interconnected cores.

are transient during the attention computation. The specific scratchpad region allocated to each matrix is determined during the inter-matrix placement phase of physical mapping. To maintain high data locality, **K**/**V** tensors are typically assigned to the same region as their corresponding weight matrices ($\mathbf{W_K}$/$\mathbf{W_V}$) in homogeneous mode. In contrast, they may be distributed across separate regions when operating in heterogeneous mode.

C. Interconnection Topology

The architecture supports both inter-tile and intra-tile connectivity. Intra-tile routers integrate local adders to perform efficient partial-sum reductions at the source as discussed in Section III-D.

1) Mesh-Based Tile: In tiles utilizing mesh interconnects, each router features five I/O ports: four for communication with adjacent routers (North, East, South, West) and one for the locally attached core, as shown in Fig. 9(a). Incoming data is buffered in dedicated FIFOs, and specialized reduction units are included to handle local reduction of partial results.

2) Tree-Based Tile: For tiles employing H-tree interconnects, the routers and links are scaled hierarchically. To match the throughput requirements of the subtrees, the link bandwidth is halved at each descending level of the hierarchy. For a subtree containing M leaf PEs, each child-to-parent downlink consists of $M/2$ parallel word lines, while the parent-to-child uplink carries M lanes, as depicted in Fig. 9(b). Likewise, incoming partial results from child links can be reduced before being sent to the next router. This H-tree structure facilitates more deterministic dataflows and reduces the area and energy overhead associated with large buffers in mesh topologies.

3) Inter-Tile Interconnect: Connectivity between two mesh-based tiles is established by linking neighboring routers along the tile boundaries. When bridging a tree-based tile and a mesh-based tile, the root router of the H-tree is interfaced with the mesh router located at the center of the adjacent boundary. In this configuration, the FIFO buffers within the mesh router manage synchronization between the two distinct tile architectures.

TABLE II
HARDWARE CONFIGURATION

Component	Specs	Component	Specs
Mesh IMC tile			
XB size	128×128	XB cell	8-bit
Rout. buf. size	32 B	Adder bitwidth	64-bit
Packet width	64-bit		
Tree IMC tile			
XB size	128×128	XB cell	8-bit
Rout. bandwidth (leaf)	64-bit	Adder bitwidth	64-bit
Packet width (leaf)	64-bit		
Mesh NMC tile			
MAC #	16	MAC bitwidth	64-bit
Scratchpad size	8 KB	Scratchpad width	64-bit

D. Communication Patterns

JADE includes the following collective communication patterns.

1) Reduction: Three reductions are involved: Reduction ❶ and ❸ are performed among the IMC outputs, and Reduction ❷ is performed among the NMC outputs, as shown in Fig. 7. The collective reduction path is implemented as a spanning tree on the mesh topology and a hierarchical tree path on the H-tree topology. The reduction is performed by dedicated adders in routers as shown in Fig. 9.

2) Broadcast: Three broadcasting steps are involved: Broadcast ❶ and ❸ are performed among the IMC inputs, and Broadcast ❷ is among the NMC inputs, as shown in Fig. 7. Likewise, the collective broadcast path is implemented as a spanning tree on the mesh topology and a hierarchical tree path on the H-tree topology. It is noted that an additional broadcast is incurred in GQA, which requires an additional broadcast of Q to the grouped K/V heads.

3) AllReduce: In the case where the crossbar array size is smaller than the width of an attention head, the reduction in the partitioned attention head is incurred. It should ideally adopt the Ring-AllReduce approach [37], [38] since this stage requires collecting the partial results in n routers and then updating them with the aggregated results. This case is not evaluated in this work, as we assume the crossbar size is fixed to 128, which aligns with the width of attention heads in Llama models.

V. RESULTS AND DISCUSSIONS

A. Experimental Setup

1) Hardware Testbed: The hardware configurations are summarized in Table II. The digital components (routers and controllers) are implemented in Verilog HDL and synthesized using Synopsys Design Compiler at a 45 nm technology node [39]. Power estimation is performed with Synopsys PrimeTime, using switching activity data from post-synthesis simulations. Scratchpad area and power are estimated via CACTI [40]. The area and power of the IMC core, featuring a 128 × 128 RRAM crossbar array, are adopted from [36].

2) Framework Implementation: The orchestration framework of JADE, including virtual mapping, physical mapping, and dataflow generator, is implemented in Python. The

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

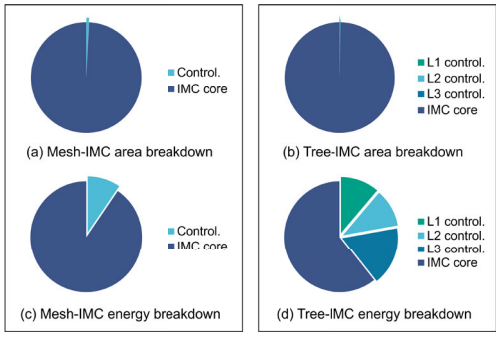


Fig. 10. Area and energy breakdown of the IMC cores in mesh and H-tree topologies. L1, L2, and L3 refer to different levels of routers in the H-tree interconnection.

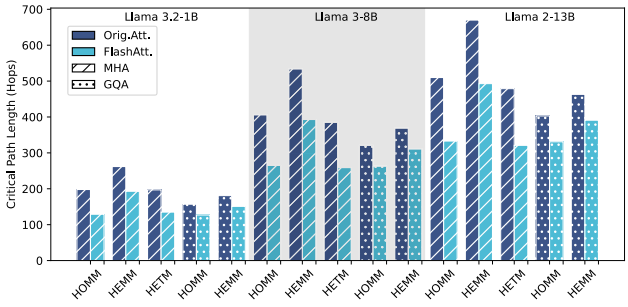


Fig. 11. Critical path of LLM inference under various architectural organizations and attention mechanisms.

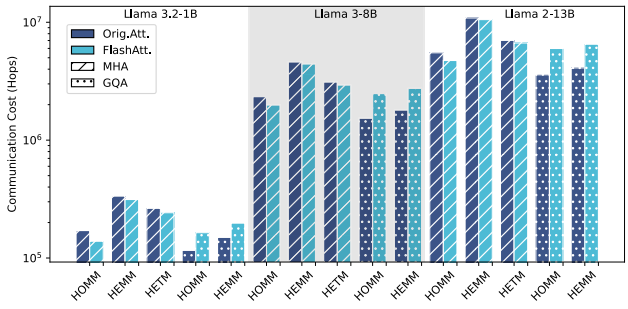


Fig. 12. Communication overhead of LLM inference under various architectural organizations and attention mechanisms.

TABLE III
COMPARISON TO GPU PLATFORMS

	Model	Ours	A100	H100
Frequency (GHz)		1	1.4	1.7
Throughput* (tokens/s)	8B - Prefill	1352.58	/	/
	8B - Decode	175.02	/	/
	8B - Overall	309.93	78.36	274.26
	13B - Prefill	1086.53	/	/
	13B - Decode	105.54	/	/
	13B - Overall	192.39	47.86	167.51
Power (W)		10.53	~300	~350
Energy efficiency (tokens/J)	8B (Llama 3)	29.44	0.2612	0.7836
	13B (Llama 2)	18.27	0.1628	0.4786

* Tested context window: 1024 input tokens, and 1024 output tokens.

specification of the digital logics and system-level scheduling are calibrated to the Verilog implementation.

3) Performance Benchmark: We evaluated the performance of the JADE using the Llama 3.2-1B [30], Llama 3-8B [28], and Llama 2-13B [27]. Our analysis focuses on three primary metrics: throughput, critical path length, and communication overhead. The evaluation results for the critical path and communication overhead are illustrated in Fig. 11 and Fig. 12, respectively. Each model was tested against ten distinct design configurations to explore the impact of architectural organizations (HOMM/HEMM/HETM), attention types (MHA and GQA), and kernel fusion options (Original vs. FlashAttention). Furthermore, Table III provides a comprehensive throughput analysis, including a detailed breakdown of performance across the prefill and decode phases. The evaluated critical path and throughput include all system-level latency components, including computation, communication, and memory access latency. The latency components are calibrated to our own Verilog implementation for digital units and to the existing work for analog units and memory units. GPU throughput reported in the manuscript includes only the execution time of LLM inference, excluding the initialization time.

B. Hardware Evaluation

The digital units for controlling and routing in the IMC and NMC tiles are implemented in Verilog HDL. The area and energy breakdown for both mesh and tree designs with 16 IMC cores are shown in Fig. 10. It shows that the logics, inclusive of the additional reduction units, account for only 1% to 3% of the macro's area, which is not a significant overhead. It takes 15% to 35% of the energy consumption, which is tolerable considering the benefit of efficient partial reduction that is provided by it.

C. Impact of Kernel Fusion

This case study assesses the impact of kernel fusion by evaluating the original attention and the FlashAttention. Compared to original attention, FlashAttention reduces the critical path significantly, as shown in Fig. 11. This stems from its kernel fusion nature, where the operations associated with the same tensors are scheduled successively. However, it has no contribution to the reduction of overall communication overhead, as shown in Fig. 12. FlashAttention was primarily proposed to reduce off-chip memory access on GPU platforms, where accessing the attention score matrix $\mathbf{S} \in \mathbb{R}^{S\times S}$ off-chip is the primary bottleneck—the loops that calculate $\mathbf{S}$ are tiled by introducing a smaller auxiliary $\mathbf{O}^* \in \mathbb{R}^{S\times D}$ to avoid calculating the entire $\mathbf{S}$. In the architecture with spatially distributed memory, this does not translate to less communication overhead since all data (either $\mathbf{S}$ or $\mathbf{O}^*$) is distributed in on-chip memory.

D. Impact of Attention Mechanism

This case study assesses the impact of two different attention mechanisms: MHA and GQA. Compared to the original MHA, GQA reduces both the critical path and communication overhead significantly, as shown in Fig. 11 and Fig. 12.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON TO LLM ACCELERATORS

		Ours	[41]	[42]	[14]
Throughput* (tokens/s)	Overall	309.93	270	36.34	N/A†
	Prefill	1352.58	N/A⋆	N/A⋆	20320.6
	Decode	175.02	N/A⋆	N/A⋆	2699.94
Energy efficiency (tokens/J)		29.44	6.8	1	N/A‡

* Estimated model sizes: 7B - 8B models.
⋆ Throughput breakdown is not reported in the articles.
† The total throughput is not explicitly reported.
‡ Energy consumption is not reported due to limited access to WSE.

This is consistent with observations on other hardware platforms. Physical mapping of Q/K/V in GQA has two possible approaches: i) Keep the same mapping as in MHA by duplicating the K/V heads; and ii) Mapping matrices into regions with different sizes that suit each of Q/K/V, where Q occupies more IMC/NMC resources due to higher data volume, and K and V are temporally multiplexed by different query heads (from Q) to perform attentions. A significant tradeoff between the memory overhead and the latency is expected in the two approaches: duplication maintains high parallelism at the cost of excessive memory overhead whereas temporal multiplexing achieves memory efficiency at the cost of increased latency. This work adopts the second approach for better memory utilization, which also translates to less communication overhead between the tensors.

E. Comparison of Prefill and Decode

This case study focuses on identifying the different bottlenecks in the prefill and decode stages. In HOMM and HEM/TMs, since the layout of the KV cache in the scratchpads is fixed, the per-token critical path has the same trend in both the prefill and decode stages. The critical path reported in Fig. 11 directly translates to the per-token latency in the decode phase, where the token-level pipeline can hardly be achieved in an autoregressive manner. However, input token-level pipelining can be achieved in the prefill stage, resulting in much higher prefill throughput and the decode phase being the bottleneck. We experimented on HOMM, which is the best case of critical path, and prefill and decode throughput in Llama 3-8B and Llama 2-13B inference are shown in Table III. The gain in prefill throughput depends on the possible number of pipelining stages. In HOMM, the token-level pipelining happens mainly in the $\mathbf{QK}^T$ and $\mathbf{S'V}$ steps, where the packet communication in the router and computation in NMC between successive tokens can be overlapped, and the maximum number of pipeline stages is equal to the number of NMC cores on its critical path. The imbalance in prefill and decode throughput has also been observed in GPUs when both stages are colocated in the computing system [43], [44], necessitating separate systems for prefill and decode, which will be explored in future research.

We further compare this work to the state-of-the-art LLM accelerators in terms of throughput and energy efficiency in Table IV. These works include wafer-scale accelerators [14] and IMC-focused work [41], [42]. This work achieves better performance compared to the previous IMC-based works, thanks to the hybrid IMC-NMC design catered for both the dynamic and static data, as well as the dedicated router design. The throughput of JADE and other IMC-based accelerators currently lags behind that of wafer-scale systems, primarily due to the vast disparity in total resource provisioning. However, assuming equivalent wafer-scale provisioning, IMC-based designs may offer a promising path toward surpassing existing fully digital wafer-scale accelerators in power efficiency. This potential advantage is largely attributed to the significant reduction in static data movement inherent in the IMC paradigm.

F. Scalability

The scalability comes in two parts: i) the time cost (*e.g.*, throughput/latency) scaling under the varying model sizes and context window lengths, and ii) the capacity limit on the maximum model sizes or context window lengths.

1) Model Size Scaling: Regarding the time cost scaling, a positive observation in this work is that the critical path does not scale up linearly to the model sizes when the context window is fixed. The critical path is increased by $\sim 2\times$ when the model size is increased by $8\times$ from Llama 3.2-1B and Llama 3-8B, as shown in Fig. 11. This stems from both how models scale and the critical path in the proposed architecture. Model size typically scales along three dimensions: the embedding dimension, the FFN hidden dimension, and the number of layers, whose scaling factors are denoted as s_e, s_h, and s_l, respectively. Under such scaling, the attention and FFN layers approximately increase in parameter count by factors of (s_e^2) and $(s_e \cdot s_h)$, respectively. When comparing Llama 3.2-1B and Llama 3-8B, for example, $s_e = 2$, $s_h = 1.75$, and $s_l = 2$, resulting in an overall model size increase of $\sim 8\times$ model. However, due to the constraints in the inter-matrix mapping, the critical path for operations such as broadcast and reduction is primarily determined by the longest horizontal or vertical communication route. As a result, the critical path scales approximately with $s_e \cdot s_l$ or $s_h \cdot s_l$ and not $s_e \cdot s_h \cdot s_l$. To validate this trend, we compare it to the results reported in [14], which show similar observations. Regarding the capacity limit, the overall IMC capacity determines the maximum amount of model parameters that can be accommodated, and the scale-out solution is necessary once the capacity is exceeded, which we leave as future work.

2) Context Window Scaling: Scaling up the context window is challenged by the capacity of the local scratchpad in NMC tiles, since the size of intermediate data in NMCs grows with the context window length. Future work might consider introducing an intermediate memory channel at the tile level to host efficient intermediate data (*e.g.*, K/V cache) loading/offloading, so that the NMC tiles would be able to transfer the K/V cache to/from the main memory by tile-level data packet hops.

VI. CONCLUSION

This work extends the IMC/NMC LLM accelerator design space with consideration of the full computing/memory/communication stacks, which enables a direct comparison between homogeneous and heterogeneous designs

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

on various levels. It provides a direct comparison between the homogeneous and heterogeneous architectural organization, which has not yet been answered in this research area. Our results reveal the strength of aggregating both IMC and NMC in the scalable homogeneous computing architectures. Computation-less attention mechanisms like GQA contribute a lot to the overall performance and computing efficiency. Regarding interconnection topology, though H-tree topology is efficient for aggregating IMCs, its applicability to NMCs and emerging attentions is poor and hence, hybrid topologies blending both mesh and tree can be beneficial. Insights are also provided that the previous kernel fusion approach may not be directly migratable to spatial accelerators with high distributed memory and dedicated hardware-software optimizations will be needed in the future.

REFERENCES

[1] S. Lie, "Cerebras architecture deep dive: First look inside the HW/SW co-design for deep learning: Cerebras systems," in *Proc. IEEE Hot Chips 34 Symp. (HCS)*, Aug. 2022, pp. 1–34.

[2] B. Chang, R. Kurian, D. Williams, and E. Quinnell, "DOJO: Supercompute system scaling for ML training," in *Proc. IEEE Hot Chips 34 Symp. (HCS)*, Aug. 2022, pp. 1–45.

[3] Y. Luo et al., "ML-CGRA: An integrated compilation framework to enable efficient machine learning acceleration on CGRAs," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.

[4] T. K. Bandara, D. Wu, R. Juneja, D. Wijerathne, T. Mitra, and L.-S. Peh, "FLEX: Introducing FLEXible execution on CGRA with spatio-temporal vector dataflow," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.

[5] X. Yang, B. Yan, H. Li, and Y. Chen, "ReTransformer: ReRAM-based processing-in-memory architecture for transformer acceleration," in *Proc. 39th Int. Conf. Comput.-Aided Design*, Nov. 2020, pp. 1–9.

[6] F. Tu et al., "TranCIM: Full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," *IEEE J. Solid-State Circuits*, vol. 58, no. 6, pp. 1798–1809, Jun. 2023.

[7] H. Li et al., "CPSAA: Accelerating sparse attention using crossbar-based processing-in-memory architecture," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 6, pp. 1741–1754, Jun. 2024.

[8] A. Jaiswal et al., "HALO: Communication-aware heterogeneous 2.5-D system for energy-efficient LLM execution at edge," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 14, no. 3, pp. 425–439, Sep. 2024.

[9] Y. Luo and S. Yu, "H3D-transformer: A heterogeneous 3D (H3D) computing platform for transformer model acceleration on edge devices," *ACM Trans. Design Autom. Electron. Syst.*, vol. 29, no. 3, pp. 1–19, May 2024.

[10] Y. Wang, Y. J. Chong, and X. Fong, "LEAP: LLM inference on scalable PIM-NoC architecture with balanced dataflow and fine-grained parallelism," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2025, pp. 1–9.

[11] M. Ge et al., "Allspark: Workload orchestration for visual transformers on processing in-memory systems," *IEEE Trans. Comput.*, vol. 74, no. 2, pp. 427–441, Feb. 2025.

[12] M. Ge, Y. Zhong, S. Chen, and Y. Kang, "Throughput maximization for transformer inference on processing near-memory architectures," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 45, no. 1, pp. 120–133, Jan. 2026.

[13] J. Zhu et al., "Theseus: Exploring efficient wafer-scale chip design for large language models," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 12, pp. 4793–4806, Dec. 2025.

[14] C. He, "WaferLLM: A wafer-scale LLM inference system," in *Proc. 19th USENIX Symp. Operating Syst. Design Implement.*, 2025, pp. 257–273.

[15] A. Ankit et al., "PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference," in *Proc. 24th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Apr. 2019, pp. 715–731.

[16] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture*. New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 754–768, doi: 10.1145/3352460.3358252.

[17] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, Jun. 2019.

[18] X. Yang et al., "Interstellar: Using halide's scheduling language to analyze DNN accelerators," in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operatings Syst.*, Mar. 2020, pp. 369–383.

[19] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1160–1174, Aug. 2021.

[20] Y. Wang, Z. Zou, and L. Zheng, "Design framework for SRAM-based computing-in-memory edge CNN accelerators," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.

[21] D. Bhattacharjee et al., "Design-technology space exploration for energy efficient AiMC-based inference acceleration," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.

[22] Y. Wang and X. Fong, "Energy-efficient Ising machines using capacitance-coupled latches for MaxCut solving," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5.

[23] S. Yang et al., "AERO: Design space exploration framework for resource-constrained CNN mapping on tile-based accelerators," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 2, pp. 508–521, Jun. 2022.

[24] Y. Wang and X. Fong, "Benchmarking DNN mapping methods for the in-memory computing accelerators," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 4, pp. 1040–1051, Dec. 2023.

[25] Y. Wang, Y. Cen, and X. Fong, "Design framework for Ising machines with bistable latch-based spins and all-to-all resistive coupling," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5.

[26] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.

[27] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[28] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.

[29] Meta.(2024). *Introducing Llama 3.1: Our Most Capable Models to Date*. Accessed: Mar. 19, 2025. [Online]. Available: https://ai.meta.com/blog/meta-llama-3-1/

[30] Meta.(2024). *Llama 3.2: Revolutionizing Edge AI and Vision With Open, Customizable Models*. Accessed: Mar. 19, 2025. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[31] T. Dao, S. Ermon, D. Fu, C. Ré, and A. Rudra, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst. 35*, 2022, pp. 16344–16359.

[32] H. Liao, J. Tu, J. Xia, and X. Zhou, "DaVinci: A scalable architecture for neural network computing," in *Proc. IEEE Hot Chips 31 Symp. (HCS)*, Aug. 2019, pp. 1–44.

[33] S. Markidis, S. W. D. Chien, E. Laure, I. B. Peng, and J. S. Vetter, "NVIDIA tensor core programmability, performance & precision," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, May 2018, pp. 522–531.

[34] T. Sharma, M. Ali, I. Chakraborty, and K. Roy, "What, when, where to compute-in-memory for efficient matrix multiplication during machine learning inference," *IEEE Trans. Emerg. Topics Comput.*, vol. 13, no. 3, pp. 1215–1229, Jul. 2025.

[35] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.

[36] X. Peng, R. Liu, and S. Yu, "Optimizing weight mapping and data flow for convolutional neural networks on processing-in-memory architectures," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 4, pp. 1333–1343, Apr. 2020.

[37] A. Gibiansky. (2017). *Bringing HPC Techniques To Deep Learning*. Accessed: Mar. 17, 2026. [Online]. Available: https://andrew.gibiansky.com/blog/machine-learning/baidu-allreduce/

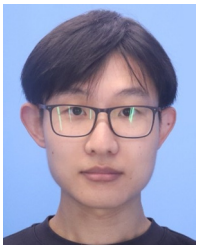
[38] A. Sergeev and M. Del Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," 2018, *arXiv:1802.05799*.

[39] J. E. Stine et al., "FreePDK: An open-source variation-aware design kit," in *Proc. IEEE Int. Conf. Microelectron. Syst. Educ. (MSE)*, San Diego, CA, USA, Jun. 2007, pp. 173–174.

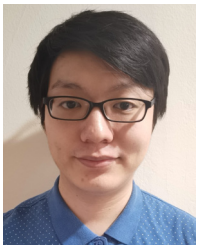
[40] N. Muralimanohar, R. Balasubramonian, and N. Jouppi, "Optimizing NUCA organizations and wiring alternatives for large caches with CACTI 6.0," in *Proc. 40th Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Dec. 2007, pp. 3–14.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.

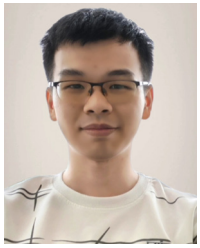
[41] M. Zhou, W. Xu, J. Kang, and T. Rosing, "TransPIM: A memory-based acceleration via software-hardware co-design for transformer," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Apr. 2022, pp. 1071–1085.
[42] Z. Yu et al., "Cambricon-LLM: A chiplet-based hybrid architecture for on-device inference of 70B LLM," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Nov. 2024, pp. 1474–1488.
[43] Y. Zhong et al., "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proc. 18th USENIX Symp. Operating Syst. Design Implement.*, 2024, pp. 193–210.
[44] P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 118–132.



Yimin Wang (Graduate Student Member, IEEE) received the B.S. degree in electronics from Jilin University (JLU), Changchun, China, in 2019. He is currently pursuing the Ph.D. degree in electrical and computer engineering with the National University of Singapore (NUS), Singapore. From 2019 to 2021, he was a Graduate student at the State Key Laboratory of Integrated Chips and Systems, Fudan University (FDU), Shanghai, China. In From 2024, he worked as a Research Intern at the Integrated Communications and AI Laboratory, Advanced Micro Devices (AMD) Research, Singapore. His current research focuses on hardware-software co-design for memory-centric architectures and domain-specific AI accelerators. He was awarded the Samsung Scholarship at Jilin University in 2018 and the Micron Foundation Prize at the National University of Singapore in 2022.



Zhen Wu (Graduate Student Member, IEEE) received the bachelor's degree from the National University of Singapore (NUS), Singapore, in 2022, where he is currently pursuing the Ph.D. degree. His research focuses on hardware acceleration for artificial intelligence, with interests in network-on-chip design, photonic integrated circuits, and scalable architectures for transformer workloads. His recent work includes hierarchical interconnect design and system-level optimization for efficient large-scale machine learning acceleration.



Yue Jiet Chong (Graduate Student Member, IEEE) received the B.Eng. degree in electrical and electronics engineering (FCH) from UTAR, Malaysia, in 2022. He is currently pursuing the Ph.D. degree with the Department of Electrical and Computer Engineering (ECE), National University of Singapore (NUS). Prior to joining NUS, he spent two years as a SoC Design Engineer at Intel Corporation. His research focuses on VLSI digital system design and hardware-software co-design of an AI accelerator.



Xuanyao Fong (Member, IEEE) received the B.Sc. (Hons.) and Ph.D. degrees in electrical engineering from Purdue University, West Lafayette, IN, USA, in 2006 and 2014, respectively.

Since December 2016, he has been an Assistant Professor at the Department of Electrical and Computer Engineering, National University of Singapore. He worked as an Intern Engineer with the Global Circuits Group (Bulldozer), Advanced Micro Devices, Inc., Boxborough Design Center, MA, USA, from January 2007 to August 2007. He was a Post-Doctoral Research Assistant at the School of Electrical and Computer Engineering, Purdue University, from September 2014 to May 2015, supervised by Prof. Kaushik Roy. Then, he joined the Institute of Microelectronics, Agency for Science, Technology and Research (A*STAR), Singapore, as a Research Scientist, in June 2015. His research interests include hardware-software co-design of scalable energy-efficient electronics and interactions across the materials-devices-circuits-architectures-algorithms stack.

Dr. Fong was a recipient of the "AMD Design Excellence Award" at Purdue in 2008. He has served on the technical program committee of ICCAD'24, EDTM'24, and BioCAS'23. He has served on the organizing committee of ISCAS'24 and CAS-related conferences/workshops in Singapore. He is an Associate Editor of IEEE JOURNAL ON EXPLORATORY SOLID-STATE COMPUTATIONAL DEVICES AND CIRCUITS, IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS, and IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:14:29 UTC from IEEE Xplore. Restrictions apply.